

Pre-Reads: OOPs Concepts - Advanced



Nishut Suman

24 Dec, 2025 At 8:00 AM

Pre-Reads

Foundation

Recommended

Resource

Pre-Reads: OOPs Concepts - Advanced



Associated Lectures (1)



Description



Discussions



Python Pre-Lecture Notes: OOPs Advance (Inheritance, super(), Method Overriding, Encapsulation, Abstraction)

Prerequisites: You should already understand **class**, **object**, **self**, and `__init__` basics.

What You'll Gain from This Pre-Read

After reading, you'll be able to:

- Understand how classes can **inherit** and **reuse** features from other classes.
- Recognize when to **override** methods for customized behavior.
- Use the **super()** function to **avoid code repetition**.
- Apply **encapsulation** to protect data from unintended access.
- Grasp how **abstraction** simplifies complex systems.

Think of this as: Learning how classes build *relationships* with each other - like families sharing traits, yet each with their own personality.

Part 1: The Big Picture – Why Does This Matter?



Opening Hook:

Have you ever noticed how your smartphone camera app feels similar across models, even if you buy a new phone? That's because each new version **inherits** core features (like **open camera**, **take photo**) from the previous one-but adds or **overrides** some behavior (like AI filters).



Connection to OOP:

In the same way, in programming, **classes can inherit** from existing ones-keeping all core functionalities while adding new or specialized ones. It saves time, keeps code organized, and helps avoid rewriting what's already working.

Where You'll Use This:

Job roles:

-  **Backend Developers:** Define reusable service classes and extend them for different APIs.
-  **Software Engineers:** Create frameworks or modules with shared logic.
-  **AI/ML Engineers:** Extend base models (e.g., **NeuralNetwork**) into specialized variants (**CNN**, **RNN**).

Real products:

-  Django's models inherit from "**models.Model**".
-  Game engines like Unity and Unreal use inheritance to define reusable **actors**.
-  Chatbots reuse base **ConversationHandler** logic in different flows.

What you can build:

- Create family-like class hierarchies (**Animal** → **Dog** → **Poodle**).
- Reuse and customize code with minimal duplication.
- Implement secure, modular systems with encapsulation.

Think of it like this: Inheritance is like **family genetics**-a child inherits traits from parents but may express them differently.

Limitation: Unlike biology, programmers have total control - children can **override** or change inherited behaviors anytime!

Part 2: Your Roadmap Through This Topic

Here's what we'll explore together:

1. Inheritance

You'll discover how one class can use attributes and methods of another, forming a parent - child relationship.

2. Method Overriding

We'll explore how child classes redefine inherited methods to behave differently.

3. `super()`

You'll see how **`super()`** helps reuse the parent class's logic without rewriting code.

4. Encapsulation

We'll discuss how to hide internal details using naming conventions (`__protected`, `__private`).

5. Abstraction

You'll understand how to simplify complex code by exposing only what's necessary.

The journey: We'll start with inheritance, then learn how to modify and protect it-finally simplifying code through abstraction.

Part 3: Key Terms to Listen For

Inheritance

A mechanism where a child class gets properties and methods from a parent class.

Example: `class Dog(Animal):` means Dog inherits from Animal.

Method Overloading

Sometimes, the *same method name* can handle **different kinds of input** - like one **tool** that works for multiple purposes.

Think of it like a **Swiss Army knife**  - one handle, but many blades for different tasks.

Method Overriding

Redefining a **parent** method in a **child** class to give it a new behavior.

Think of it as: Replacing a song's remix version over the original.

Key Takeaway:

- **Overloading** → Same method name, but works with **different numbers or types of inputs** (usually within the same class).
 - **Overriding** → Same method name, but shows **different behavior in a child class**.
-

super()

A built-in function that lets a child class access methods of its parent class.

In practice: Used to call the parent's `__init__` without retyping everything.

Encapsulation

Restricting direct access to an object's internal data.

Example: Like a bank account, you can deposit or withdraw money, but you can't open the vault directly. In Python, methods like **deposit()** and **get_balance()** protect the private data (e.g., **__balance**) and control how it's accessed.

Abstraction

Exposing only essential features while hiding unnecessary complexity.

Example: When you call **open("file.txt")**, you don't need to know how the operating system actually reads or writes the file.

💡 **Key Insight:** Together, these principles make code more **reusable, secure, and readable** - just like building with Lego blocks instead of clay.

Part 4: Concepts in Action

🧩 Seeing Inheritance in Action

The scenario: Let's imagine we're modeling animals in a zoo. All animals can **speak**, but each does it differently.

Our approach: We'll define a base class **Animal**, then inherit and override behavior in subclasses.

```
# Step 1: Define the parent class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Some generic animal sound")

# Step 2: Child class inherits Animal
class Dog(Animal):
    def speak(self):
        print("Woof!") # Overriding parent's method

# Step 3: Another child class
class Cat(Animal):
    def speak(self):
        print("Meow!") # Overriding parent's method

# Step 4: Create objects
dog = Dog("Buddy")
cat = Cat("Whiskers")
```

```
dog.speak() # Output: Woof!  
cat.speak() # Output: Meow!
```

What's happening here:

- Both **Dog** and **Cat** inherit from **Animal**.
- Each child redefines (**overrides**) the **speak()** method.
- The parent's generic method is replaced by specific ones.

The output/result:

```
Woof!  
Meow!
```

Key takeaway: Inheritance lets us reuse structure, and overriding lets us customize behavior.

Using super() in Action

```
class Student:  
    def __init__(self, name, id):  
        self.name = name  
        self.id = id  
  
class GraduateStudent(Student):  
    def __init__(self, name, id, thesis_topic):  
        super().__init__(name, id) # Calls Student's __init__  
        self.thesis_topic = thesis_topic  
  
    def introduce(self):  
        print(f"Hi, I'm {self.name}, working on '{self.thesis_topic}'.")  
  
grad = GraduateStudent("Aditi", 42, "AI Ethics")  
grad.introduce()
```

Output:

```
Hi, I'm Aditi, working on 'AI Ethics'.
```

Key takeaway:

super() lets us build upon parent logic cleanly - no repetition.

Encapsulation & Abstraction Example

```
class BankAccount:  
    def __init__(self, owner, balance):  
        self.owner = owner
```

```
self.__balance = balance # Private attribute

def deposit(self, amount):
    self.__balance += amount

def get_balance(self):
    return self.__balance
```

Conceptual Flow:

- **__balance** can't be accessed directly (encapsulation).
- We expose only **deposit()** and **get_balance()** (abstraction).

Image Reference:

 [Encapsulation Diagram \(GeeksforGeeks\)](#)

 [OOP Hierarchy Visual](#)

Part 5: How This Topic Connects

Builds on:

- Classes and Objects
- **__init__** and **self**

Enables:

- Building real-world models (e.g., Students, Employees, Vehicles)
- Writing maintainable codebases
- Designing modular and scalable systems

Related concepts you might explore:

- **Polymorphism:** How one function can behave differently for different objects.
- **Composition:** Using objects of other classes instead of inheriting.
- **Access Modifiers:** Python's naming conventions for visibility.

Part 6: Questions to Keep in Mind

1. Why do we need inheritance if we can just copy-paste code?
2. What's the real difference between overriding and overloading?
3. How does encapsulation improve program security and maintainability?

Reflect: Think of a real-life system (like a school or company). How would you design its classes using inheritance?

Quick Self-Check: Did It Click?

- ☐ I can explain how inheritance reduces code duplication.
 - ☐ I understand how **super()** helps call parent methods.
 - ☐ I can recognize the use of encapsulation and abstraction in real-world APIs.
 - ☐ I can identify overridden methods in a class hierarchy.
-

How to Read These Notes Effectively

- ✓ Read actively-visualize relationships like family trees.
 - ✓ Try running the small code snippets.
 - ✓ Look for patterns (“what’s reused?” “what’s customized?”).
 - ✗ Don’t try to memorize everything-focus on understanding relationships.
-

Additional resources:

- [Python Docs: **super\(\)** Function](#)
 - [Real Python: Inheritance and Composition](#)
 - [Programiz: Python Inheritance](#)
-

Final Thought

You’re about to unlock some of Python’s *true OOP superpowers*. Inheritance, overriding, encapsulation, and abstraction aren’t just theory-they’re what make professional-grade software *reusable, secure, and elegant*.

You’re ready to dive deeper into OOP Superpowers! 🚀
